

UNIVERSIDADE DE MOGI DAS CRUZES

ENGENHARIA DE SOFTWARE

DIEGO VICTORIO BENETTI DE SOUSA

HUDSON ALVES DOS SANTOS

WILLIAN OLIVEIRA DOS SANTOS

PROJETO BANCO DE DADOS

Eventpay: Vendas, divulgação e organização de eventos diversos

MOGI DAS CRUZES

2025

DIEGO VICTORIO BENETTI DE SOUSA

HUDSON ALVES DOS SANTOS

WILLIAN OLIVEIRA DOS SANTOS

PROJETO BANCO DE DADOS

Eventpay: Vendas, divulgação e organização de eventos diversos

Trabalho Acadêmico apresentado
ao curso de Engenharia de
Software da Universidade de Mogi
das Cruzes como parte dos
requisitos para obtenção de nota.

Prof. Orientador(a): Terigi Augusto
Scardovelli

MOGI DAS CRUZES

2025

1 INTRODUÇÃO	3
1.1 TEMA DO PROJETO.....	3
1.2 EQUIPE	3
1.3 OBJETIVO.....	3
1.4 DESCRIÇÃO DO MINIMUNDO.....	4
1.5 REQUISITOS FUNCIONAIS	4
1.6 REQUISITOS NÃO FUNCIONAIS	10
1.7 REGRAS DE NEGÓCIO.....	15
2 DESENVOLVIMENTO	16
2.1 DER.....	16
2.2 MODELO LÓGICO	17
2.3 DICIONÁRIO DO BANCO DE DADOS	18
3 SCRIPT DE CRIAÇÃO DO BANCO DE DADOS E DAS TABELAS	22
4 SCRIPT DE POVOAMENTO DE TODAS AS TABELAS.....	25
5 CRIAÇÃO DE PROCEDURES	30
6 CRIAÇÃO DAS VIEWS	41
7 CRIAÇÃO DOS ÍNDICES	42
8 CRIAÇÃO DA TRANSAÇÃO	42

1 INTRODUÇÃO

1.1 TEMA DO PROJETO

Desenvolvimento de um Website de Vendas, Divulgação e Organização de Eventos diversos

1.2 EQUIPE

Diego Victorio Benetti de Sousa, Hudson Alves dos santos e William oliveira dos santos

1.3 OBJETIVO

O objetivo deste projeto é modelar e implementar um banco de dados relacional para um Website de Vendas, Divulgação e Organização de Eventos diversos. A proposta é desenvolver uma estrutura capaz de armazenar, organizar e controlar informações relacionadas aos eventos, clientes, compras, ingressos, organizadores e demais operações do sistema.

1.4 DESCRIÇÃO DO MINIMUNDO

O sistema será um website de divulgação, organização e venda de ingressos para eventos. Nele, empresas se cadastram para criar e gerenciar seus eventos, informando dados como nome, CNPJ, contatos e endereço. Cada evento possui nome, data, hora, descrição, gênero, capacidade e endereço próprio. Um evento oferece um ou mais tipos de ingressos, cada um com preço e status. Clientes também se cadastram no site com seus dados pessoais, endereço e telefones. Eles podem comprar ingressos de diferentes eventos, registrando quantidade adquirida e status da compra. Cada compra gera um pagamento, que registra valor, data e quantidade de ingressos, e está associado a uma forma de pagamento (como cartão, boleto ou Pix). O sistema controla a disponibilidade de ingressos, garante que não ultrapasse a capacidade do evento e permite que empresas e clientes acompanhem suas informações e transações dentro da plataforma.

1.5 REQUISITOS FUNCIONAIS

Tela Cadastro	RF01 - O cliente pode se cadastrar na plataforma com suas credenciais: nome, Email, senha, CPF, data de nascimento, telefone e endereço, ou com sua conta do Google. (RF41,RD01,RD04)
Tela Login	RF02 - O cliente pode se se logar no sistema com suas credenciais: Email e senha, ou com sua conta do Google; (RF41,RD02) RF03 - O cliente pode solicitar a recuperação de senha. (RF42, RNF02)
Tela Home	RF04 - O cliente pode visualizar seu nome, Email, eventos disponíveis, calendário, carrinho e seu perfil. (RF46, RF47, RF63, RNF03, RE01)
Tela Perfil	RF05 - O cliente poderá visualizar nome, Email; (RF63, RNF03, RNF08,RD01) RF06 - O cliente pode acessar as configurações de perfil. (RF42)
Tela Carrinho	RF07 - O cliente pode visualizar os ingressos salvos; (RF62, RNF04, RNF05, RNF06) RF08 - O cliente pode finalizar (fechar o carrinho) o seu pedido; (RF61, RNF04, RNF07) RF09 - O cliente pode excluir os ingressos. (RF60, RNF06)

Tela calendário	RF10 - O cliente pode visualizar os dias disponíveis dos eventos, data, hora e local do evento. (RF46, RF47, RF52, RNF03, RNF09, RD01, RE04)
Tela Evento	RF11 - O cliente pode visualizar o nome, local, data, hora e descrição do evento; (RF47, RF52, RNF03, RD01) RF12 - O cliente pode favoritar o evento; (RF56, RE03) RF13 - O cliente pode realizar a busca de um evento em específico (RF43, RF53, RE02) RF14 - O cliente pode compartilhar o evento; (RF57, RNF10) RF15 - O cliente pode adicionar os ingressos ao carrinho. (RF59, RF54)
Tela Configurações	RF16 - O cliente pode deletar a sua conta; (RF42, RE05) RF17 - O cliente pode fazer o logout da plataforma; (RF41, RE05) RF18 - O cliente pode alterar seus dados como: Email, nome, senha, telefone e endereço. (RF42, RNF01, RNF08, RNF13)
Tela Compra	RF19 - O cliente pode visualizar e alterar a quantidade de ingressos; (RF58, RF62, RF48, RF54, RE06) RF20 - O cliente pode realizar a solicitação de pagamento. (RF45, RF54, RE06)

--	--

Tela Cadastro	RF21 - A empresa pode se cadastrar com nome, e-mail, CNPJ, senha, telefone e endereço. (RF41,RD04)
Tela Login	RF22 - A empresa pode se logar no sistema com suas credenciais: CNPJ e senha; (RF41, RD02) RF23 - A empresa pode recuperar a senha. (RF42, RNF14)
Tela Home	RF24 - A empresa pode visualizar o nome, CNPJ, calendário, seu perfil, estatísticas e perfil dos clientes; (RF46, RF47, RF51, RF63, RE07) RF25 - A empresa pode adicionar eventos. (RF57)
Tela Perfil	RF26 - A empresa pode visualizar o nome, email, telefone, endereço, sobre a empresa CNPJ e suas publicações; (RF63, RNF18, RE07) RF27 - A empresa pode acessar as configurações de perfil. (RNF11, RE07)
Tela Eventos	RF28 - A empresa pode adicionar eventos informando o nome do evento, imagem, data, local, horário, descrição, gênero e o valor a ser pago; (RF47, RF52, RF57, RNF11, RNF12, RNF21, RF44, RD09, RE07) RF29 - A empresa pode compartilhar o evento; (RF57)

	RF30 - A empresa pode adicionar vários eventos. (RF57, RNF12)
Tela Dados	RF31 - A empresa pode adicionar os dados da sua conta bancária: número da agência, número da conta, chave pix da empresa; (RNF15, RD10, RE07, RE08) RF32 - A empresa pode alterar os dados bancários: chave pix da empresa; (RF42) RF33 - A empresa pode visualizar os pagamentos realizados. (RF50, RNF16)
Tela Calendário	RF34 - A empresa pode acessar o calendário; (RF46, RNF17, RE07) RF35 - A empresa pode visualizar os dias vagos para realizar o evento e visualizar os eventos fornecidos por outras empresas. (RF46, RF47, RNF17, RE09)
Tela Estatísticas	RF36 - A empresa pode visualizar as suas estatísticas e o ranking. (RF51, RD06, RE07)
Tela Configurações	RF37 - A empresa pode deletar a sua conta; (RF42) RF38 - A empresa pode fazer o logout da plataforma; (RF42) RF39 - A empresa pode alterar seus dados como: Email, nome, senha, telefone e endereço. (RF42)

Tela Histórico	RF40 - A empresa pode visualizar o seu histórico de eventos, os eventos realizados e os futuros eventos e feedbacks. (RF49, RF50, RF46, RNF16, RNF18,RD06)
-----------------------	--

REQUISITOS FUNCIONAIS DO SISTEMA

RF41 - O sistema permitirá que o cliente e empresa faça cadastro, login e logout. (RF01, RF02, RF03,RF17, RF21, RF22,RD01,RD02)
RF42 – O sistema permitirá que o usuário e empresa altere e delete suas informações. (RF03, RF06, RF16, RF18, RF23, RF28, RF32, RF37, RF38, RF39)
RF43 - o sistema permite a pesquisa de posts. (RF13)
RF44 - o sistema permite adicionar, deletar e alterar posts. (RF28)
RF45 - o sistema permite fazer pagamento de ingressos dentro da plataforma. (RF20, RD10)
RF46 - o sistema permite acessar o calendário. (RF04, RF10, RF24, RF34, RF35, RF40)
RF47 - o sistema permite consultar eventos e horários disponíveis. (RF04, RF10, RF11, RF24, RF28, RF35)
RF48 - o sistema permite que o cliente cancele a compra de ingressos. (RF19)
RF49 - o sistema permite a visualização de histórico de pesquisas. (RF40, RD05)
RF50 - o sistema permite a visualização do histórico de compra de ingressos. (RF33, RF40, RD05)

RF51 - o sistema permite a avaliação do serviço contratado. (RF24, RF36)
RF52 - o sistema permite a geolocalização. (RF10, RF11, RF28, RD07)
RF53 - o sistema permite filtrar posts (RF13)
RF54 - o sistema permite a compra de ingressos. (RF15, RF19, RF20, RD11)
RF55 - o sistema permite a comunicação entre o usuário e a empresa
RF56 - o sistema permite o recebimento de notificações. (RF12,RD03,RD08)
RF57 - o sistema permite divulgar eventos. (RF13, RF24,RF28, RF29)
RF58 - o sistema permite alterar a quantidade de ingressos. (RF18)
RF59 - o sistema deve adicionar ingressos ao carrinho. (RF14)
RF60 - o sistema deve excluir ingressos do carrinho. (RF09)
RF61 - o sistema permite finalizar a compra de ingressos (fechar carrinho). (RF08, RD11)
RF62 - o sistema permite visualizar os produtos do carrinho. (RF07, RF18)
RF63 - o sistema irá exibir os perfis cadastrados para os demais usuários. (RF04, RF05, RF23, RF25)

1.6 REQUISITOS NÃO FUNCIONAIS

RNF01 - O cliente não pode alterar o seu CPF da plataforma (RF18)

RNF02 - O cliente só poderá pedir a recuperação da senha, se possuir um Email ativo na plataforma (RF03)

RNF 03 - O cliente não poderá ter acesso a informações pessoais de outros usuários; (RF04, RF05, RF10, RF11)

RNF 04 - O cliente não pode ter acesso ao carrinho de outros usuários;(RF07, RF08)

RNF 05 - O cliente não pode acessar os produtos de outros usuários. (RNF 04, RF07)

RNF 06 - O cliente não pode excluir, adicionar e favoritar os ingressos de outros usuários; (RNF 05, RNF 04, RF09, RF07)

RNF 07 - O cliente não pode cancelar pedidos de outros usuários; (RNF06, RF08)

RNF 08 - O cliente não tem acesso à senha e a conta de outros usuários; (RNF 03, RF05)

RNF 09 - O cliente não pode ter acesso a outros calendários; (RF10)

RNF 10 - O cliente não pode realizar a venda de ingressos. (RF13)

RNF 11 – A empresa só pode adicionar arquivos do tipo, jpeg, png, svg e mp4;(RF28)

RNF 12 - A empresa não pode gerenciar o post de outras empresas;(RF27, RF29)

RNF 13 - A empresa não pode alterar o CNPJ;(RF17)

RNF 14 - A empresa só poderá alterar a senha mediante a um Email valido; (RNF 02)

RNF 15 - A empresa não pode adicionar mais de uma conta bancária na plataforma;(RF30)

RNF 16 - A empresa não pode visualizar o extrato e o histórico de outras contas;(RF32, RF39)

RNF 17 - a empresa não pode ver o calendário com os eventos marcados de outras contas;(RF33, RF34)

RNF 18 - a empresa não tem acesso as outras contas e nem a dados pessoais de outras contas;(RF25)

RNF 19 - a empresa não pode ter acesso ao chat de conversas de outros usuários;(RF39)

RNF 20 - a empresa deve ter acesso a internet para acessar a plataforma;

RNF 21 - a empresa só pode adicionar um ingresso para compra, mediante ao preenchimento de todos os dados (uma descrição detalhada sobre o ingresso). (RF28)

DESEMPENHO	O sistema deve ter um tempo de resposta de, no máximo, 3 segundos para as funcionalidades gerais. O tempo de resposta para o processamento de pagamento de ingressos deve ser de, no máximo, 7 segundos. O sistema deve ter tempo de resposta para atualização das informações de no máximo 5 segundos.
-------------------	--

	O sistema deve ter latência de no máximo 100 milissegundos;
DISPONIBILIDADE	O sistema deve permanecer disponível 24 horas por dia, 7 dias por semana, exceto em períodos de manutenção. A interface deve ser intuitiva e acessível para todos os tipos de usuários (clientes e empresas). O sistema deve funcionar em computadores, smartphones e tablets. O sistema pode ter até 4 minutos de inatividade não programada por semana O sistema deve ser capaz de se recuperar de uma falha e voltar a ficar operacional em menos de 1 hora;
	O sistema não deve permitir que um usuário comum acesse a área de pagamento ou de agendamento de eventos. O sistema deve utilizar criptografia para senhas e para pagamentos de ingressos.

SEGURANÇA	O sistema deve conter criptografia nos dados dos usuários. O sistema deve ser resistente a ataques comuns, como DDoS e outros tipos de invasão. O sistema deve bloquear a conta do usuário após 10 tentativas de login com senha incorreta. O sistema deve conter controles de acesso baseados em funções ou outros mecanismos de autorização para garantir que os usuários só possam acessar os dados e funcionalidades que têm permissão para usar;
CONFIABILIDADE	O sistema deve monitorar falhas críticas do sistema Tempo médio entre falhas (superior a 2 horas) Tempo médio para reparação (inferior a 1 hora)
ROBUSTEZ	O sistema deve garantir mensagens de erro claras e explicativas

	O sistema deve ser resistente a falhas do hardware O sistema deve ser capaz de recuperar dados após falha de sistema
MANUTENIBILIDADE	As manutenções do sistema devem ter duração máxima de 30 minutos. O sistema deve ter baixa complexidade para manutenção
ESCALABILIDADE	A adição de novos recursos ao sistema deve exigir no máximo 20% de esforço adicional em relação ao esforço do desenvolvimento original. O sistema deve suportar um aumento de 50% na quantidade de usuários sem perda de funcionalidades. O sistema deve suportar um aumento de 100% no volume de dados sem perda ou alteração das informações. O sistema deve suportar até 1500 usuários síncronos sem perda de desempenho ou funcionalidades.

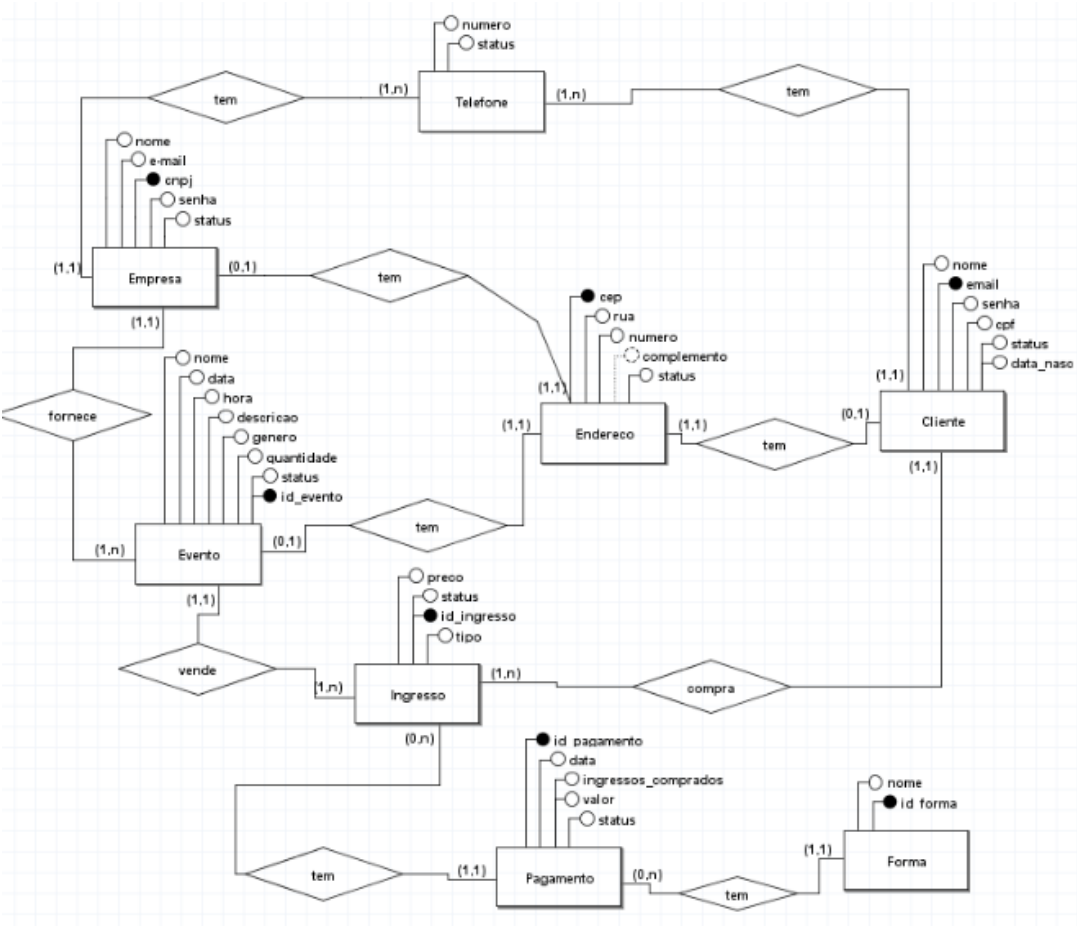
1.7 REGRAS DE NEGÓCIO

REGRAS GERAIS	Todo cliente devera para efetivar o cadastro individual, conter um CPF válido e dar os aceites nos termos e condições. Toda empresa deve possuir um cadastro individual, CNPJ ativo e válido.
SENHA	A senha deve possuir no mínimo 8 caracteres contendo no mínimo uma letra maiúscula, um número e um caractere especial. O sistema deve bloquear o login após 5 tentativas falhas de acesso.
PAGAMENTO E VENDAS	Cada ingresso possui código único e é atribuído ao e-mail do cliente.
EXECUÇÃO	Clientes só podem solicitar reembolso até 48 horas antes do evento
POLÍTICAS	Nenhum evento pode ser criado com data anterior à data atual.

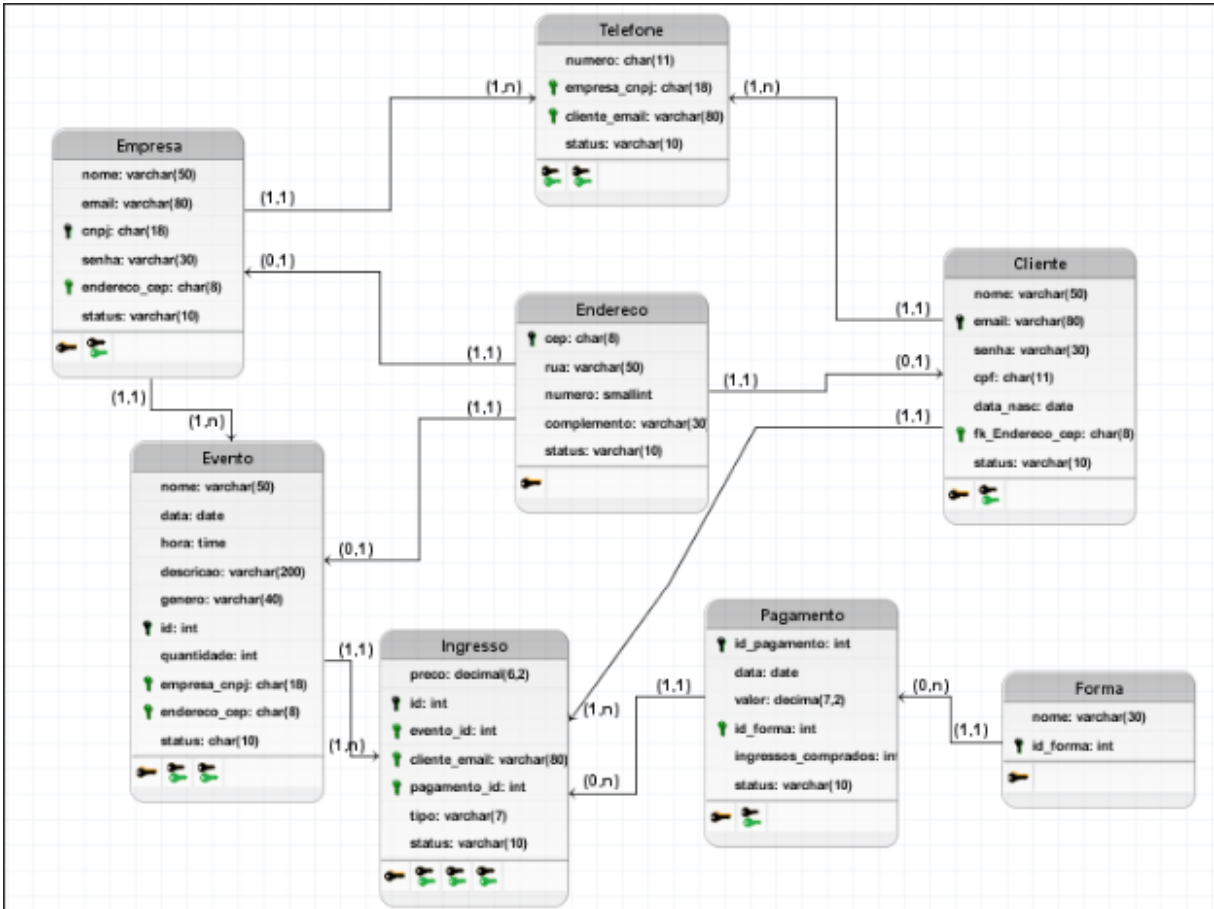
	Usuários não podem comprar mais ingressos do que o limite definido pelo organizador.
--	---

2 DESENVOLVIMENTO

2.1 DER



2.2 MODELO LÓGICO



2.3 DICIONÁRIO DO BANCO DE DADOS

TABELA	Endereco			
DESCRIÇÃO	Armazenará as informações de endereço			
OBSERVAÇÕES				
	CAMPOS			
Nome	Descrição	Tipo de dado	Tamanho	Restrições de domínio
cep	Codigo do endereço	char	8	PK
rua	Nome da rua	varchar	50	Not Null
numero	Número do local	smallint		Not Null
complemento	Complemento do local	varchar	30	
status	Status atual da tabela se está ativo ou inativo	varchar	10	Not Null

TABELA	Cliente			
DESCRIÇÃO	Armazenará as informações do cliente			
OBSERVAÇÕES	Essa tabela possui uma chave estrangeira da tabela Endereco			
	CAMPOS			
Nome	Descrição	Tipo de dado	Tamanho	Restrições de domínio
nome	Nome do cliente	varchar	50	Not Null
email	Email do cliente	varchar	80	PK
senha	Senha da conta do cliente	smallint		Not Null
cpf	Codigo de pessoa	char	11	Unique
dt_nasc	Data de nascimento do cliente	date		Not Null
cep	Codigo do endereço	char	8	FK
status	Status atual da tabela se está ativo ou inativo	varchar	10	Not Null

TABELA	Empresa			
DESCRIÇÃO	Armazenará as informações da empresa			
OBSERVAÇÕES	Essa tabela possui uma chave estrangeira da tabela Endereco			
	CAMPOS			
Nome	Descrição	Tipo de dado	Tamanho	Restrições de domínio
nome	Nome da Empresa	varchar	50	Not Null
email	Email da empresa	varchar	80	Unique
cnpj	Codigo de identificação da empresa	char	18	PK
senha	Senha da conta	varchar	30	Not Null
cep	Chave estrangeira referenciada da tabela Endereco	char	8	FK
status	Status atual da tabela se está ativo ou inativo	varchar	10	Not Null

TABELA	Pagamento
--------	-----------

DESCRIÇÃO	Armazenará as informações do pagamento			
OBSERVAÇÕES	Essa tabela possui uma chave estrangeira da tabela Forma			
	CAMPOS			
Nome	Descrição	Tipo de dado	Tamanho	Restrições de domínio
id_pagamento	Codigo de identificação do pagamento	int		PK
data	Data do pagamento	datetime		Not Null
valor	Valor do pagamento total	decimal	7,2	Not Null
id_forma	Chave estrangeira referenciada da tabela forma	int		FK
ingressos_comprados	Quantidade de ingressos comprados pelo cliente	int		Not Null
status	Status atual da tabela se está ativo ou inativo	varchar	10	Not Null

TABELA	Forma			
DESCRIÇÃO	Armazenará as informações das formas de pagamentos			
	CAMPOS			
Nome	Descrição	Tipo de dado	Tamanho	Restrições de domínio
id_forma	Identificação da forma de pagamento	int		PK
Nome	Nome da forma de pagamento	varchar	30	Not Null

TABELA	Telefone			
DESCRIÇÃO	Armazenará as informações de número de telefone			
OBSERVAÇÕES	Essa tabela possui uma chave estrangeira da tabela cliente e da tabela empresa			
	CAMPOS			
Nome	Descrição	Tipo de dado	Tamanho	Restrições de domínio
numero	Numero de telefone da empresa ou cliente	int		FK
cnpj	Chave estrangeira referenciada da tabela empresa	char	11	PK
email	Chave estrangeira referenciada da tabela cliente	varchar	11	Not Null
status	Status atual da tabela se está ativo ou inativo	varchar	10	Not Null

TABELA	Evento			
DESCRIÇÃO	Armazenará as informações do evento			
OBSERVAÇÕES	Essa tabela possui uma chave estrangeira da tabela Endereco e da tabela empresa			
	CAMPOS			
Nome	Descrição	Tipo de dado	Tamanho	Restrições de dominio
nome	Nome do evento	varchar	50	Not Null
data	Data do evento	date		Not Null
hora	Horario da realização do evento	time		Not Null
descricao	Descrição do evento	varchar	200	Not Null
genero	Tipo de evento	varchar	40	Not Null
id_evento	Codigo do evento	int		PK
quantidade	Quantidade de ingressos disponiveis para o evento	int		Not Null
cnpj	Chave estrangeira referenciada da tabela empresa	char	11	FK
cep	Chave estrangeira referenciada da tabela endereco	char	8	FK
status	Status atual da tabela se está ativo ou inativo	varchar	10	Not Null

TABELA	Ingresso			
DESCRIÇÃO	Armazenará as informações do Ingresso			
OBSERVAÇÕES	Essa tabela possui uma chave estrangeira da tabela cliente, pagamento e evento			
	CAMPOS			
Nome	Descrição	Tipo de dado	Tamanho	Restrições de dominio
preco	Valor unitario do ingresso	decimal	6,2	Not Null
id_ingresso	Codigo do ingresso	int		PK
id_evento	Chave estrangeira referenciada da tabela evento	int		FK
email	Chave estrangeira referenciada da tabela cliente	varchar	11	FK
id_pagamento	Chave estrangeira referenciada da tabela pagamento	int		FK
tipo	Tipo do ingresso podendo ser inteira ou meia	varchar	7	Not Null
status	Status atual da tabela se está ativo ou inativo	varchar	10	Not Null

3 SCRIPT DE CRIAÇÃO DO BANCO DE DADOS E DAS TABELAS

-- Criação do Banco de Dados --

```
create database eventpay;
```

```
use eventpay;
```

-- Criação das Tabelas --

```
create table Endereco (  
cep char(8) not null,  
rua varchar(50) not null,  
numero smallint not null check (numero > 0),  
complemento varchar(30),  
status varchar(10) not null default 'ativo',  
constraint pk_endereco primary key(cep)  
);
```

```
create table forma(  
id int auto_increment,  
nome varchar(30) not null check( nome = 'pix' or nome = 'credito' or nome = 'debito' or  
nome = 'boleto'),  
constraint pk_forma_id_forma primary key(id)  
);
```

```
create table pagamento(  
id int auto_increment,  
data datetime not null default current_timestamp,  
valor decimal(7,2) not null,  
id_forma int not null,
```

```
ingressos_comprados int not null,  
status varchar(10) not null default 'ativo',  
constraint pk_id primary key(id),  
constraint fk_pagamento_id_forma foreign key(id_forma) references forma(id) on  
update cascade on delete restrict
```

```
);
```

```
create table empresa(  
cnpj char(18),  
nome varchar(50) not null,  
email varchar(80) unique not null,  
senha varchar(30) not null check (CHAR_LENGTH(senha) >= 8),  
endereco_cep char(8),  
status varchar(10) not null default 'ativo',  
constraint pk_cnpj primary key(cnpj),  
constraint fk_empresa_endereco_cep foreign key(endereco_cep) references  
endereco(cep) on update cascade on delete set null  
);
```

```
create table evento(  
id int auto_increment,  
nome varchar(50) not null,  
data date not null,  
hora time not null,  
descricao varchar(200) not null,  
genero varchar(40) not null,  
quantidade int not null,  
empresa_cnpj varchar(18),  
endereco_cep char(8),
```

```
status varchar(10) not null default 'ativo',
constraint pk_id primary key(id),
constraint fk_evento_empresa_cnpj foreign key(empresa_cnpj) references
empresa(cnpj) on delete restrict,
constraint fk_evento_endereco_cep foreign key(endereco_cep) references
endereco(cep) on update cascade on delete set null
);
```

```
create table cliente(
email varchar(80),
nome varchar(50) not null,
senha varchar(30) not null check (CHAR_LENGTH(senha) >= 8),
cpf char(11) unique,
data_nasc date not null,
endereco_cep char(8),
status varchar(10) not null default 'ativo',
constraint pk_cliente primary key(email),
constraint fk_cliente_endereco_cep foreign key(endereco_cep) references
endereco(cep) on update cascade on delete set null
);
```

```
create table telefone(
numero char(11) not null,
empresa_cnpj char(18),
cliente_email varchar(80),
status varchar(10) not null default 'ativo',
constraint pk_numero primary key(numero),
constraint fk_empresa_cnpj foreign key(empresa_cnpj) references empresa(cnpj) on
delete set null,
```



```
constraint fk_cliente_email foreign key(cliente_email) references cliente(email) on
update cascade on delete cascade
);
```

```
create table ingresso(
id int auto_increment,
tipo varchar(7) not null check(tipo = "Inteira" or tipo = "Meia"),
preco decimal(6,2) not null,
evento_id int,
cliente_email varchar(80),
pagamento_id int,
status varchar(10) not null default 'ativo',
constraint pk_id primary key(id),
constraint fk_ingresso_evento_id foreign key(evento_id) references evento(id) on
update cascade on delete restrict,
constraint fk_ingresso_cliente_email foreign key(cliente_email) references
cliente(email) on update cascade on delete set null,
constraint fk_ingresso_pagamento_id foreign key(pagamento_id) references
pagamento(id) on update cascade on delete set null
);
```

4 SCRIPT DE POVOAMENTO DE TODAS AS TABELAS

-- Criação dos Inserts

```
insert into endereco(cep,rua,numero,complemento) values
('01001000','Av. Paulista',1578,'Conj. 101'),
('02022030','Rua Voluntários da Pátria',345,'Apto 12'),
('04045040','Rua Vergueiro',2122,'Bloco B'),
```

```
('05010020','Rua Cerro Corá',800,'Casa'),
('06018025','Rua Antônio Agu',120,'Sala 3'),
('07090015','Av. Lucas Nogueira Garcez',500,'Loja 1'),
('08015010','Rua das Palmeiras',45,'Fundos'),
('09030000','Av. Getúlio Vargas',900,'Apto 21'),
('11045060','Rua XV de Novembro',310,'Casa'),
('13023055','Rua Barão de Jaguará',1500,'Sala 4');
```

insert into forma(nome) values

```
('pix'),
('credito'),
('debito'),
('boleto'),
('pix'),
('credito'),
('debito'),
('boleto'),
('pix'),
('credito');
```

insert into empresa(cnpj,nome,email,senha,endereco_cep) values

```
('12.345.678/0001-90','TechWave
Solutions','contato@techwave.com','senha123','01001000'),
('23.456.789/0001-80','Brasil
Ltda','eventos@brasil.com','segura12','02022030'),
('34.567.890/0001-70','Cia
Aurora','contato@aurorateatro.com','senha999','04045040'),
('45.678.901/0001-60','Festival
Music','info@brmusic.com','music123','05010020'),
```

Eventos

Teatro

Brasil

('56.789.012/0001-50','Gastrô
Gourmet','contato@gastrofeiras.com','gast1234','06018025'), Feira
(('67.890.123/0001-40','GameWorld
Expo','contato@gameworld.com','expo2024','07090015'),
(('78.901.234/0001-30','Balada Neon Club','neon@club.com','neon7777','08015010'),
(('89.012.345/0001-20','Anime
Convention','contato@animebr.com','otaku123','09030000'), Brasil
(('90.123.456/0001-10','EcoFeira Sustentável','eco@feira.com','verde456','11045060'),
(('11.987.654/0001-00','TechMasters
Conference','info@techmasters.com','tech2025','13023055'));

insert cliente(email,nome,senha,cpf,data_nasc,endereco_cep) values

('ana.silva@gmail.com','Ana Silva','ana12345','12345678901','1990-03-12','01001000'),
(('joao.mendes@yahoo.com','João Mendes','jmendes22','98765432100','1988-07-20','02022030'),
(('mariana.rocha@outlook.com','Mariana Rocha','mar12345','11122233344','1995-01-25','04045040'),
(('carlos.souza@gmail.com','Carlos Souza','souza555','22233344455','1992-04-18','05010020'),
(('juliana.lima@hotmail.com','Juliana Lima','lima9090','33344455566','1998-10-10','06018025'),
(('ricardo.pires@gmail.com','Ricardo Pires','rp202456','44455566677','1985-11-05','07090015'),
(('fernanda.alves@uol.com','Fernanda Alves','123fern7','55566677788','1991-09-09','08015010'),
(('thiago.santos@gmail.com','Thiago Santos','ts777777','66677788899','1996-06-30','09030000'),
(('isabela.castro@gmail.com','Isabela Castro','isa12345','77788899900','1994-02-14','11045060'),
(('marcos.paiva@gmail.com','Marcos Paiva','mar11555','88899900011','1987-12-01','13023055'));

insert into telefone(numero,empresa_cnpj,cliente_email) values

('11990010001','12.345.678/0001-90',NULL),
('11990010002','23.456.789/0001-80',NULL),
('11990010003','34.567.890/0001-70',NULL),
('11990010004','45.678.901/0001-60',NULL),
('11990010005','56.789.012/0001-50',NULL),
('11980020001',NULL,'ana.silva@gmail.com'),
('11980020002',NULL,'joao.mendes@yahoo.com'),
('11980020003',NULL,'mariana.rocha@outlook.com'),
('11980020004',NULL,'carlos.souza@gmail.com'),
('11980020005',NULL,'juliana.lima@hotmail.com');

insert into pagamento(data,valor,id_forma,ingressos_comprados) values

('2025-01-01',89.90,1,10),
('2025-01-02',120.00,2,2),
('2025-01-03',150.50,3,28),
('2025-01-04',200.00,4,2),
('2025-01-05',75.00,5,3),
('2025-01-06',95.00,6,8),
('2025-01-07',110.00,7,10),
('2025-01-08',180.00,8,19),
('2025-01-09',220.00,9,9),
('2025-01-10',60.00,10,13);

INSERT INTO evento(nome, data, hora, descricao, genero, quantidade, empresa_cnpj, endereco_cep) VALUES

('Show Capital Inicial', '2025-03-10', '20:00:00', 'Show da banda Capital Inicial com repertório dos maiores sucessos.', 'Show', 3000, '12.345.678/0001-90', '01001000'),
('Musical O Rei Leão', '2025-04-12', '18:00:00', 'Espetáculo musical inspirado na famosa produção da Broadway.', 'Teatro', 1500, '23.456.789/0001-80', '02022030'),

('Stand Up com Thiago Ventura', '2025-05-01', '19:30:00', 'Show de comédia stand-up com o humorista Thiago Ventura.', 'Comédia', 800, '34.567.890/0001-70', '04045040'),

('Festival de Jazz São Paulo', '2025-06-22', '21:00:00', 'Festival musical reunindo grandes artistas nacionais e internacionais do jazz.', 'Show', 5000, '45.678.901/0001-60', '05010020'),

('Feira Internacional de Gastronomia', '2025-07-15', '17:00:00', 'Feira com chefs renomados e degustações de diversas culinárias do mundo.', 'Feira', 2000, '56.789.012/0001-50', '06018025'),

('GameWorld Expo 2025', '2025-08-10', '10:00:00', 'Evento de games com lançamentos, campeonatos e área para testes.', 'Expo', 6000, '67.890.123/0001-40', '07090015'),

('Neon Night Festival', '2025-09-05', '23:00:00', 'Festival noturno com música eletrônica, efeitos de luz e DJs renomados.', 'Show', 3500, '78.901.234/0001-30', '08015010'),

('Anime Brasil 2025', '2025-10-12', '09:00:00', 'Grande encontro para fãs de anime, mangá, cosplay e cultura pop japonesa.', 'Expo', 8000, '89.012.345/0001-20', '09030000'),

('EcoFeira Sustentável 2025', '2025-11-01', '08:00:00', 'Feira dedicada à sustentabilidade, reciclagem e projetos ecológicos.', 'Feira', 1500, '90.123.456/0001-10', '11045060'),

('TechMasters Conference 2025', '2025-12-20', '14:00:00', 'Conferência tecnológica com palestras sobre inovação, IA e futuro digital.', 'Palestra', 1200, '11.987.654/0001-00', '13023055');

insert into ingresso(tipo,preco,evento_id,cliente_email,pagamento_id) values

('Inteira',120.00,1,'ana.silva@gmail.com',1),

('Inteira',150.00,2,'joao.mendes@yahoo.com',2),

('Meia',180.00,3,'mariana.rocha@outlook.com',3),

('Inteira',250.00,4,'carlos.souza@gmail.com',4),

('Meia',80.00,5,'juliana.lima@hotmail.com',5),

('Meia',300.00,6,'ricardo.pires@gmail.com',6),

('Inteira',90.00,7,'fernanda.alves@uol.com',7),

('Meia',250.00,8,'thiago.santos@gmail.com',8),

('Inteira',70.00,9,'isabela.castro@gmail.com',9),

('Inteira',350.00,10,'marcos.paiva@gmail.com',10);

5 CRIAÇÃO DE PROCEDURES

-- Criação dos Procedures --

-- Exemplo de Chamada --

-- select * from endereco;

-- call insert_endereco('07500000','Av. Paulista',1578,'Conj. 101');

-- Procedures Endereco --

delimiter //

create procedure insert_endereco(

in p_cep char(8),

in p_rua varchar(50),

in p_numero smallint,

in p_complemento varchar(30)

)

begin

insert into endereco(cep,rua,numero,complemento) values (p_cep, p_rua,
p_numero, p_complemento);

end //

create procedure update_endereco(

in p_cep char(8),

in p_rua varchar(50),

in p_numero smallint,

in p_complemento varchar(30)

)

begin

```
        update endereco set rua = p_rua, numero = p_numero, complemento =  
p_complemento where cep = p_cep;  
end //
```

```
create procedure delete_endereco(  
in p_cep char(8)  
)  
begin  
    update endereco  
    set status = 'inativo'  
    where cep = p_cep;  
end //
```

```
delimiter ;
```

```
-- Chamando a Procedure --
```

```
CALL insert_endereco('99999999', 'Rua Exemplo', 100, 'Casa A');  
CALL update_endereco('99999999', 'Rua Alterada', 120, 'Casa B');  
CALL delete_endereco('99999999');  
select * from endereco;
```

```
-- Procedures Forma --
```

```
delimiter //
```

```
create procedure insert_forma(in p_nome varchar(30))  
begin  
    insert into forma(nome) values(p_nome);  
end //
```

```
create procedure update_forma(in p_id int, in p_nome varchar(30))
begin
    update forma set nome = p_nome where id = p_id;
end //
```

```
create procedure delete_forma(in p_id int)
begin
    delete from forma where id = p_id;
end //
```

```
delimiter ;
```

```
-- Chamando a Procedure --
```

```
CALL insert_forma('pix');
```

```
CALL update_forma(1, 'credito');
```

```
CALL delete_forma(1);
```

```
-- Procedures Empresa --
```

```
delimiter //
```

```
create procedure insert_empresa(
    in p_cnpj char(18),
    in p_nome varchar(50),
    in p_email varchar(80),
    in p_senha varchar(30),
    in p_endereco_cep char(8)
```



```
)  
begin  
    insert into empresa(cnpj,nome,email,senha,endereco_cep) values (p_cnpj,  
p_nome, p_email, p_senha, p_endereco_cep);  
end //
```

```
create procedure update_empresa(  
    in p_cnpj char(18),  
    in p_nome varchar(50),  
    in p_email varchar(80),  
    in p_senha varchar(30),  
    in p_endereco_cep char(8)  
)  
begin  
    update empresa  
        set nome = p_nome, email = p_email, senha = p_senha, endereco_cep =  
p_endereco_cep where cnpj = p_cnpj;  
end //
```

```
create procedure delete_empresa(in p_cnpj char(18))  
begin  
    update empresa  
        set status = 'inativo'  
        where cnpj = p_cnpj;  
end //  
desc empresa;  
delimiter ;
```

```
-- Chamando a Procedure --
```

```
CALL insert_empresa('12.000.000/0001-00', 'Nova Empresa', 'empresa@teste.com',  
'senha1234', '01001000');
```

```
CALL update_empresa('12.000.000/0001-00', 'Empresa Nova Ltda',  
'contato@empresa.com', 'novaSenha88', '02022030');
```

```
CALL delete_empresa('12.000.000/0001-00');
```

```
select * from empresa;
```

```
-- Procedures Evento --
```

```
delimiter //
```

```
create procedure insert_evento(  
    in p_nome varchar(50),  
    in p_data date,  
    in p_hora time,  
    in p_descricao varchar(200),  
    in p_genero varchar(40),  
    in p_quantidade int,  
    in p_empresa_cnpj char(18),  
    in p_endereco_cep char(8)  
)
```

```
begin
```

```
    insert into evento(nome, data, hora, descricao, genero, quantidade,  
empresa_cnpj, endereco_cep)
```

```
    values(p_nome, p_data, p_hora, p_descricao, p_genero, p_quantidade,  
p_empresa_cnpj, p_endereco_cep);
```

```
end //
```

```
create procedure update_evento(  
    in p_id int,  
    in p_nome varchar(50),
```

```

        in p_data date,
        in p_hora time,
        in p_descricao varchar(200),
        in p_genero varchar(40),
        in p_quantidade int,
        in p_empresa_cnpj char(18),
        in p_endereco_cep char(8)
    )
begin
update evento
set nome = p_nome,
    data = p_data,
    hora = p_hora,
    descricao = p_descricao,
    genero = p_genero,
    quantidade = p_quantidade,
    empresa_cnpj = p_empresa_cnpj,
    endereco_cep = p_endereco_cep
where id = p_id;
end //

create procedure delete_evento(in p_id int)
begin
    update evento
    set status = 'inativo'
    where id = p_id;
end //

```

delimiter ;

-- Chamando a Procedure --

CALL insert_evento(

'Festa Teste', '2025-05-10', '19:00:00', 'Descrição exemplo', 'Show', 500,
'12.345.678/0001-90', '01001000');

CALL update_evento(

1, 'Evento Alterado', '2025-06-01', '20:00:00', 'Descrição atualizada', 'Teatro', 3000,
'23.456.789/0001-80', '02022030'

);

CALL delete_evento(1);

select * from evento;

-- Procedure Pagamento --

delimiter //

create procedure insert_pagamento(

in p_data date,

in p_valor decimal(7,2),

in p_id_forma int,

in p_ingressos_comprados int

)

begin

insert into pagamento(data, valor, id_forma, ingressos_comprados) values
(p_data, p_valor, p_id_forma, p_ingressos_comprados);

end //

```
create procedure update_pagamento(  
    in p_id int,  
    in p_data date,  
    in p_valor decimal(7,2),  
    in p_id_forma int,  
    in p_ingressos_comprados int  
)  
begin  
    update pagamento  
    set data = p_data,  
    valor = p_valor,  
    id_forma = p_id_forma,  
    ingressos_comprados = p_ingressos_comprados  
    where id = p_id;  
end //
```

```
create procedure delete_pagamento(in p_id int)  
begin  
    update pagamento  
    set status = 'inativo'  
    where id = p_id;  
end //
```

```
delimiter ;
```

```
-- Chamando a Procedure
```

```
CALL insert_pagamento('2025-05-01', 150.00, 1, 2);
```

```
CALL update_pagamento(1, '2025-05-02', 200.00, 2, 3);
```

```
CALL delete_pagamento(1);
```

```
select * from pagamento
```

```
-- Procedure Cliente --
```

```
delimiter //
```

```
create procedure insert_cliente(
```

```
    in p_email varchar(80),
```

```
    in p_nome varchar(50),
```

```
    in p_senha varchar(30),
```

```
    in p_cpf char(11),
```

```
    in p_data_nasc date,
```

```
    in p_cep char(8)
```

```
)
```

```
begin
```

```
    insert into cliente(email, nome, senha, cpf, data_nasc, endereco_cep) values  
    (p_email, p_nome, p_senha, p_cpf, p_data_nasc, p_cep);
```

```
end //
```

```
create procedure update_cliente(
```

```
    in p_email varchar(80),
```

```
    in p_nome varchar(50),
```

```
    in p_senha varchar(30),
```

```
    in p_cpf char(11),
```

```
    in p_data_nasc date,
```

```
    in p_cep char(8)
```

```
)
```

```
begin
```

```
update cliente
set nome = p_nome,
senha = p_senha,
cpf = p_cpf,
data_nasc = p_data_nasc,
endereco_cep = p_cep
where email = p_email;
end //
```

```
create procedure delete_cliente(in p_email varchar(80))
begin
update cliente
set status = 'inativo'
where email = p_email;
end //
```

```
delimiter ;
```

```
-- Chamando a Procedure --
```

```
CALL insert_cliente('teste@teste.com', 'Cliente Teste', 'senha1234', '12345678999',
'1990-01-01', '01001000');
```

```
CALL update_cliente('teste@teste.com', 'Cliente Alterado', 'senha9999',
'999999999999', '1991-02-02', '02022030');
```

```
CALL delete_cliente('teste@teste.com');
```

```
select * from cliente;
```

```
-- Procedure Telefone --
```

```
delimiter //
```

```
create procedure insert_telefone(  
    in p_numero char(11),  
    in p_empresa_cnpj char(18),  
    in p_cliente_email varchar(80)  
)  
begin  
    insert into telefone(numero, empresa_cnpj, cliente_email)  
    values (p_numero, p_empresa_cnpj, p_cliente_email);  
end //
```

```
create procedure update_telefone(  
    in p_numero char(11),  
    in p_empresa_cnpj char(18),  
    in p_cliente_email varchar(80)  
)  
begin  
    update telefone  
    set empresa_cnpj = p_empresa_cnpj,  
    cliente_email = p_cliente_email  
    where numero = p_numero;  
end //
```

```
create procedure delete_telefone(in p_numero char(11))  
begin  
    update telefone  
    set status = 'inativo'  
    where numero = p_numero;  
end //
```


delimiter;

-- Chamando a Procedure --

CALL insert_telefone('11999999999', '12.345.678/0001-90', NULL);

CALL update_telefone('11999999999', NULL, 'ana.silva@gmail.com');

CALL delete_telefone('11999999999');

select * from telefone;

-- Procedure Ingresso --

delimiter //

create procedure insert_ingresso(

in p_tipo varchar(50),

in p_preco decimal(6,2),

in p_evento_id int,

in p_cliente_email varchar(80),

in p_pagamento_id int

)

begin

insert into ingresso(tipo, preco, evento_id, cliente_email, pagamento_id) values
(p_tipo, p_preco, p_evento_id, p_cliente_email, p_pagamento_id);

end //

create procedure update_ingresso(

in p_id int,

in p_tipo varchar(50),

in p_preco decimal(6,2),

in p_evento_id int,

```
        in p_cliente_email varchar(80),
        in p_pagamento_id int
    )
begin
    update ingresso
    set tipo = p_tipo,
        preco = p_preco,
        evento_id = p_evento_id,
        cliente_email = p_cliente_email,
        pagamento_id = p_pagamento_id
    where id = p_id;
end //
```

```
create procedure delete_ingresso(in p_id int)
begin
    update ingresso
    set status = 'inativo'
    where id = p_id;
end //
```

```
delimiter ;
```

```
-- Chamando a Procedure --
```

```
CALL insert_ingresso('Inteira', 150.00, 1, 'ana.silva@gmail.com', 1);
```

```
CALL update_ingresso(1, 'Meia', 80.00, 2, 'joao.mendes@yahoo.com', 2);
```

```
CALL delete_ingresso(1);
```

```
select * from ingresso;
```

6 CRIAÇÃO DAS VIEWS

-- Criação das Views --

create view view_eventos_disponiveis as

```
select evento.id,  
       evento.nome,  
       evento.data,  
       evento.hora,  
       evento.descricao,  
       evento.genero,  
       evento.quantidade as capacidade_total
```

from evento

left join ingresso on ingresso.evento_id = evento.id

group by evento.id;

SELECT * FROM view_eventos_disponiveis;

create view view_compra as

```
select evento.nome,  
       ingresso.tipo as tipo_ingresso,  
       ingresso.preco as preco_unitario,  
       pagamento.ingressos_comprados,  
       (ingresso.preco * pagamento.ingressos_comprados) as valor_total
```

from ingresso

left join evento on evento.id = ingresso.evento_id

left join pagamento on pagamento.id = ingresso.pagamento_id;

SELECT * FROM view_compra;

```
create view view_carrinho_usuario as
select cliente.email,
       cliente.nome,
       empresa.nome as nome_empresa,
       count(ingresso.id) as total_itens,
       sum(ingresso.preco) as total_carrinho
from cliente
left join ingresso on ingresso.cliente_email = cliente.email
left join evento on evento.id = ingresso.evento_id
left join empresa
       on empresa.cnpj = evento.empresa_cnpj
group by cliente.email;
SELECT * FROM view_carrinho_usuario;
```

7 CRIAÇÃO DOS ÍNDICES

-- Criação dos Indices --

```
create fulltext index idx_evento_resumo on evento(nome,descricao);
create index idx_ingresso_preco on ingresso(preco);
create index idx_empresa_nome on empresa(nome);
```

8 CRIAÇÃO DA TRANSAÇÃO

-- Criação da Transação --

```
delimiter //
create procedure comprar_ingresso(
       in p_evento_id int,
```

```
        in p_qtd_comprada int,
        in p_valor_total decimal(10,2),
        in p_forma int
    )
begin
    declare estoque_atual int;

    start transaction;

    select quantidade into estoque_atual
    from evento
    where id = p_evento_id;

    if estoque_atual < p_qtd_comprada then
        rollback;
    end if;

    insert into pagamento (valor, id_forma, ingressos_comprados)
    values (p_valor_total, p_forma, p_qtd_comprada);

    update evento
    set quantidade = quantidade - p_qtd_comprada
    where id = p_evento_id;

    commit;
end //

delimiter ;

-- Transação Exemplo --
```

```
-- select * from pagamento;  
-- select * from evento;  
-- CALL comprar_ingresso(1, 1, 150.00, 1);
```